



Experiment 08: Automating Application Deployment using GitHub Actions / Jenkins

Experiment No.	08
Mapped CO	CO3
Aim	To implement an automated deployment pipeline using GitHub Actions or Jenkins.
Tools / Software	GitHub account / Jenkins; Docker Hub or a container registry; A deployment target such as Kubernetes or a server

Theory

Continuous Deployment (CD) is the practice of automatically releasing every validated software change to the target environment. After the CI stages such as build and test are successful, the deployment stage packages the application, publishes the build artifact or container image, and updates the running environment.

In a container-based deployment pipeline, the workflow commonly logs in to a container registry, builds a Docker image, tags it with a meaningful version such as a commit SHA, pushes it to Docker Hub or another registry, and then triggers deployment on Kubernetes or a remote server.

GitHub Actions implements deployment as code through YAML workflow files. Jenkins provides a similar automation mechanism using Pipeline jobs and Jenkinsfiles. Secrets such as Docker credentials, cluster credentials and server keys must be stored securely as GitHub repository secrets or Jenkins credentials and must never be committed to source control.

Procedure

1. Create or use an existing GitHub repository containing the application source code and Dockerfile.
2. Ensure the CI workflow already builds and tests the application successfully.
3. Create a Docker Hub account or use another container registry for storing the application image.
4. Store registry credentials securely as repository secrets, for example DOCKER_USERNAME and DOCKER_PASSWORD.
5. Create or update the workflow file in .github/workflows, for example deploy.yml or ci.yml.
6. Add a deployment job that runs only after the build-test job is successful and only on the main branch.
7. Add steps to check out the source code, log in to Docker Hub, build the Docker image and push it with a versioned tag.
8. Add a deployment step to update the target environment, for example by using kubectl set image for Kubernetes or a remote server restart command.
9. Commit and push the workflow file to the main branch.
10. Open the Actions tab or Jenkins build history and verify that the build, image push and deployment stages complete successfully.
11. Confirm that the latest deployed application is running by opening the deployment URL or executing a curl command.

Sample Code / Commands

GitHub Actions deployment job appended to ci.yml

```
deploy:
  needs: build-test
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: docker/login-action@v3
      with:
        username: ${ secrets.DOCKER_USERNAME }
        password: ${ secrets.DOCKER_PASSWORD }
```



```
- uses: docker/build-push-action@v6
  with:
    push: true
    tags: ${{ secrets.DOCKER_USERNAME }}/myapp:${{ github.sha }}
```

Example deployment update commands

```
# Kubernetes target
kubectl set image deployment/myapp myapp=${{ secrets.DOCKER_USERNAME }}/myapp:${{ github.sha }}
kubectl rollout status deployment/myapp

# Server target example
ssh user@server 'docker pull <user>/myapp:<tag> && docker restart myapp'
```

Observation / Experiment Output

Instructions to students: Paste screenshots or copied outputs in the boxes below after completing the automated deployment pipeline using GitHub Actions or Jenkins. Keep screenshots clear and readable.

Name	Roll No.	Batch	Date
Pushkar SHinde	58	B3	

1. Repository and Application Setup Output

Paste screenshot/output showing GitHub repository, application source code, Dockerfile and CI workflow file location.

2. Repository Secrets / Jenkins Credentials Output

Paste screenshot/output showing configured repository secrets or Jenkins credentials. Do not reveal actual secret values.

3. Deployment Workflow YAML Output

Paste screenshot/output showing deploy job with build-test dependency, registry login, image build and image push steps.



The screenshot shows the GitHub Actions interface for the repository 'Pushkar3232 / DevOps-APSIT'. The 'Actions' tab is selected, displaying a list of workflow runs. A notification at the top indicates an upcoming change to the GitHub App installation token format. The workflow runs are listed in a table with columns for Workflow, Event, Status, Branch, and Actor. The first two runs are for the workflow 'ci: add Docker continuous deployment pipeline' on the 'main' branch, both with a status of 'Success'. The third run is for the workflow 'feat: add Docker deployment version 2' on the 'main' branch, also with a status of 'Success'.

Workflow	Event	Status	Branch	Actor
ci: add Docker continuous deployment pipeline	CD #5: Commit c9f1d3c pushed by Pushkar3232	Success	main	Pushkar3232
ci: add Docker continuous deployment pipeline	CI #5: Commit c9f1d3c pushed by Pushkar3232	Success	main	Pushkar3232
feat: add Docker deployment version 2	CI #4: Commit a9efe66 pushed by Pushkar3232	Success	main	Pushkar3232

4. Pipeline Execution Output

Paste screenshot/output showing successful GitHub Actions/Jenkins pipeline stages for build, test, image push and deployment.

The screenshot shows a pipeline execution output for a workflow named 'cd.yml' triggered by a 'push' event. The pipeline consists of two stages: 'build-test' and 'docker'. Both stages are marked as successful with green checkmarks. The 'build-test' stage took 13 seconds to complete, and the 'docker' stage took 26 seconds to complete.

```
cd.yml
on: push

build-test:
  # ...

docker:
  # ...
```

5. Container Registry Output

Paste screenshot/output showing the pushed Docker image/tag in Docker Hub or the selected container registry.

The screenshot shows a container registry output for a pushed Docker image. The image is named 'devops-apsit' and has a tag of '2.0'. The image is located at '8080:5000' and has a size of '0.01%'. The image was pushed 1 hour ago.

Image	Tag	Location	Size	Pushed
devops-apsit	2.0	8080:5000	0.01%	1 hour ago

6. Deployment Verification Output

Paste screenshot/output showing Kubernetes rollout/server deployment status and application response from browser or curl.



← CD
✓ ci: add Docker continuous deployment pipeline #5 Re-run all jobs Latest #3

Summary
All jobs

Annotations
1 warning

build-test
✓ docker

Run details
Usage
Workflow file

docker
succeeded 2 minutes ago in 26s

Search logs

- Set up job 1s
- Checkout code 1s
- Log in to Docker Hub 1s
- Build Docker image 12s
- Push Docker image 7s
- Post Log in to Docker Hub 1s
- Post Checkout code 0s
- Complete job 0s

GitHub Repository Link: <https://github.com/Pushkar3232/DevOps-APSIT>

GitHub Actions / Jenkins Pipeline Link: <https://github.com/Pushkar3232/DevOps-APSIT/actions/runs/32258376845/job/96088004682>

Docker Hub / Container Registry Image Link:
<https://hub.docker.com/repository/docker/pushkar654/devops-apsit/general>

Deployment URL / Endpoint:

Final Observation Statement:

The CI/CD pipeline was successfully configured to test the application, build the Docker image, and push it to Docker Hub automatically.
The workflow successfully connected GitHub Actions with Docker Hub for automated image deployment.

Expected Output

After tests pass on the main branch, the workflow should automatically build the application image, tag it, push it to the container registry and deploy it to the selected target environment. The final application should be accessible through the deployment URL or endpoint.

Conclusion

An automated deployment pipeline was implemented using GitHub Actions or Jenkins. The experiment demonstrated how validated code changes can be packaged, published and deployed automatically, completing a commit-to-deploy DevOps workflow.

Viva / Review Questions

1. How are secrets handled securely in a CD pipeline?
2. What is a rolling update and why is it preferred?
3. Why should Docker images be tagged with a commit SHA instead of only latest?
4. Differentiate Continuous Delivery and Continuous Deployment.



PARSHWANATH CHARITABLE TRUST'S

A.P. SHAH INSTITUTE OF TECHNOLOGY

Department of Computer Science and Engineering
Data Science



Marks: ____ / 10

Date: _____ Signature of Faculty: _____